

Logging in Python

An Explanatory Guide with Practical Examples

This guide explains what logging is, why it matters more than `print()` once code leaves your laptop, and how to set it up correctly — with runnable examples throughout. As before, examples follow **ShopEasy**, a retail company running data pipelines across Indian cities.

1. What Is Logging, and Why Not Just Use `print()`?

Definition: Logging is the practice of recording messages about a program's execution — what it's doing, what went wrong, and when — to a destination (console, file, or external system) that persists beyond the program's run, and that can be filtered by importance.

`print()` works fine while you're writing code on your own machine. It breaks down the moment code runs unattended — a nightly ETL job, a scheduled Airflow task, an API server — because nobody is watching the console when it fails at 2 AM.

<code>print()</code>	logging
Disappears when the terminal closes	Can be written permanently to a file or log system
No severity levels — every message looks the same	Built-in levels (DEBUG, INFO, WARNING, ERROR, CRITICAL) let you filter noise from real problems
Always runs, everywhere, whether you want it or not	Can be turned up or down per module without touching the code
No timestamp, no source, no context by default	Automatically includes timestamp, module name, severity, and more
Hard to disable across a large codebase	Centrally configurable — one config line changes behaviour everywhere

A quick before/after

```
# Before -- print debugging
print("Starting order import...")
print("Order ORD-4521 failed: invalid pincode")

# After -- logging
import logging
logger = logging.getLogger(__name__)

logger.info("Starting order import...")
logger.error("Order ORD-4521 failed: invalid pincode")
```

The second version can be redirected to a file, filtered to show only errors in production, and automatically tagged with a timestamp and the exact module that logged it — none of which `print()` gives you for free.

2. The Five Logging Levels

Every log message has a severity level. This is the single most useful feature of logging: it lets you leave detailed messages in your code permanently, then decide later — without editing the code — how much of it you actually want to see.

DEBUG (level 10)

Definition: *Fine-grained, detailed information useful only while actively diagnosing a problem.*

Use it when: you want to trace exactly what values a variable held, or exactly which branch of an if/else the code took — detail you don't want cluttering normal output.

ShopEasy scenario: Printing the exact API payload sent to the pincode validation service, for use only while debugging a failing integration.

```
logger.debug("Validating pincode payload: %s", payload)
```

INFO (level 20)

Definition: *Confirmation that things are working as expected — the normal, healthy heartbeat of the program.*

Use it when: you want a record of major milestones in a process: started, completed, N records processed.

ShopEasy scenario: Recording that the nightly order-import job started and how many orders it processed.

```
logger.info("Order import started for %s", import_date)
logger.info("Order import completed: %d orders processed", order_count)
```

WARNING (level 30)

Definition: *Something unexpected happened, or a potential problem was noticed, but the program can still continue running normally.*

Use it when: data looks off but isn't fatal — a fallback was used, a value was out of the expected range, a deprecated feature was called.

ShopEasy scenario: An order arrives with a missing delivery city, so the pipeline defaults it to “Unassigned” and continues.

```
logger.warning("Order %s missing city, defaulting to Unassigned", order_id)
```

ERROR (level 40)

Definition: *A serious problem happened — the program failed to complete a specific task or operation, though it may still be able to continue with other work.*

Use it when: an operation genuinely failed: a file couldn't be read, a database write failed, an API call returned an error.

ShopEasy scenario: A single order record fails validation and is skipped, but the rest of the batch keeps processing.

```
logger.error("Failed to import order %s: invalid pincode format", order_id)
```

CRITICAL (level 50)

Definition: *A very serious error — the program itself may be unable to continue running at all.*

Use it when: the failure is so severe the whole application or process cannot proceed — a database connection pool is completely exhausted, a required config file is missing.

ShopEasy scenario: The pipeline cannot connect to the orders database at all, so the entire nightly job must abort.

```
logger.critical("Cannot connect to orders database -- aborting import job")
```

Practical takeaway: the levels have numeric values (DEBUG=10 ... CRITICAL=50) precisely so you can set a threshold. Setting the logger to WARNING means DEBUG and INFO messages are silently dropped, with zero code changes.

3. Core Components: Logger, Handler, Formatter

Python's logging module has three moving parts. Understanding the difference between them removes almost all the confusion beginners have.

Component	Job
Logger	The object your code calls (<code>logger.info(...)</code>). Decides <i>whether</i> a message is important enough to process at all, based on its level.
Handler	Decides <i>where</i> an accepted message goes — console, a file, email, a remote log server. A single logger can have several handlers at once.
Formatter	Decides <i>what the message looks like</i> — whether it includes a timestamp, the module name, the severity label, etc.

Think of it like a courier company: the **Logger** decides whether a parcel (message) is worth sending at all, the **Handler** decides which depot it goes to (console, file, email), and the **Formatter** decides what the shipping label looks like.

How they connect

```
import logging

logger = logging.getLogger("shopeasy.orders")    # 1. Logger -- your entry point
logger.setLevel(logging.INFO)                    # only INFO and above are processed

handler = logging.StreamHandler()                # 2. Handler -- console output
handler.setLevel(logging.INFO)

formatter = logging.Formatter(                   # 3. Formatter -- message layout
    "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
)
handler.setFormatter(formatter)

logger.addHandler(handler)

logger.info("Order import started")
logger.debug("This will NOT show -- logger level is INFO")

2026-09-06 10:15:32,101 | INFO      | shopeasy.orders | Order import started
```

4. Quick Setup: logging.basicConfig()

For simple scripts, you don't need to wire up Logger/Handler/Formatter by hand — `basicConfig()` does it in one call. This is the right choice for small scripts and early-stage projects.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(message)s"
)

logging.info("ShopEasy ETL job started")
logging.warning("Order ORD-889: missing pincode, using default")
logging.error("Order ORD-901: could not be saved to database")
```

```
2026-09-06 10:20:01,004 | INFO      | ShopEasy ETL job started
2026-09-06 10:20:01,006 | WARNING   | Order ORD-889: missing pincode, using default
2026-09-06 10:20:01,009 | ERROR    | Order ORD-901: could not be saved to database
```

Watch out: `basicConfig()` only has effect the **FIRST** time it's called in a program. Calling it again later, or after another part of your code already logged something, silently does nothing. This trips up a lot of beginners.

Getting a named logger per module (the recommended pattern)

In any file/module larger than a quick script, use `logging.getLogger(__name__)` instead of the root logger directly. This automatically names each logger after its module (`shopeasy.pipeline.orders`, `shopeasy.pipeline.customers`, etc.), so log messages tell you exactly which part of the codebase produced them.

```
# In shopeasy/pipeline/orders.py
import logging
logger = logging.getLogger(__name__)

def import_orders(date):
    logger.info("Importing orders for %s", date)
```

5. Logging to a File and the Console at Once

A very common real-world setup: show INFO and above on the console for a developer watching the job run, while writing everything (including DEBUG) to a file for later inspection.

```
import logging

logger = logging.getLogger("shopeasy.pipeline")
logger.setLevel(logging.DEBUG)  # logger itself allows everything through

# Handler 1: console, INFO and above only
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.INFO)
console_handler.setFormatter(logging.Formatter("%(levelname)s: %(message)s"))

# Handler 2: file, EVERYTHING including DEBUG
file_handler = logging.FileHandler("shopeasy_pipeline.log")
file_handler.setLevel(logging.DEBUG)
file_handler.setFormatter(logging.Formatter(
    "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
))

logger.addHandler(console_handler)
logger.addHandler(file_handler)

logger.debug("Connecting to orders database")  # file only
logger.info("Order import started")           # both console and file
```

Practical takeaway: each handler can have its own level and its own formatter. This is how production systems keep the console clean while still capturing full detail in a file for post-mortem debugging.

6. Formatting Log Messages

The format string controls what each log line looks like. These are the placeholders you'll use most often.

Placeholder	Shows	Example
<code>%(asctime)s</code>	Timestamp of the log call	2026-09-06 10:20:01,004
<code>%(levelname)s</code>	Severity level as text	WARNING
<code>%(name)s</code>	Name of the logger (usually the module)	shopeasy.pipeline.orders
<code>%(message)s</code>	The actual log message	Order import started
<code>%(filename)s</code>	Source file name	orders.py
<code>%(lineno)d</code>	Line number of the log call	42
<code>%(funcName)s</code>	Function the log call was made in	import_orders

Practical takeaway: always pass variables as arguments -- `logger.info("Order %s failed", order_id)` -- rather than an f-string like `logger.info(f"Order {order_id} failed")`. With the argument form, Python only builds the final string if the message actually gets logged; with an f-string, it's built every time, even if that log level is disabled.

7. Log Rotation: Keeping Log Files Under Control

A pipeline that runs every night will produce a log file that grows forever unless you rotate it. Python's logging module has two built-in handlers for this.

RotatingFileHandler -- rotate by file size

```
from logging.handlers import RotatingFileHandler
import logging

handler = RotatingFileHandler(
    "shopeasy_pipeline.log",
    maxBytes=5_000_000,    # rotate once the file hits ~5 MB
    backupCount=3          # keep 3 old versions: .log.1, .log.2, .log.3
)
handler.setFormatter(logging.Formatter("%(asctime)s | %(levelname)s | %(message)s"))

logger = logging.getLogger("shopeasy.pipeline")
logger.addHandler(handler)
logger.setLevel(logging.INFO)
```

TimedRotatingFileHandler -- rotate on a schedule

```
from logging.handlers import TimedRotatingFileHandler

handler = TimedRotatingFileHandler(
    "shopeasy_pipeline.log",
    when="midnight",      # roll over to a new file every day at midnight
    backupCount=7         # keep the last 7 days
)
```

Watch out: without rotation, a daily pipeline log can silently grow to gigabytes over months and eventually fill the disk -- this is a real production incident, not a theoretical one.

8. Logging Exceptions Properly

When something throws an exception, logging just the message loses the traceback — the exact information you need to actually fix the bug. Use `logger.exception()` or pass `exc_info=True` instead.

```
import logging
logger = logging.getLogger(__name__)

def save_order(order):
    try:
        write_to_database(order)
    except ConnectionError:
        # WRONG -- loses the traceback entirely
        logger.error("Failed to save order")

        # RIGHT -- automatically includes the full traceback
        logger.exception("Failed to save order %s", order["id"])

        # Equivalent alternative, useful outside an except block
        logger.error("Failed to save order %s", order["id"], exc_info=True)
```

```
ERROR:shopeasy.orders:Failed to save order ORD-4521
Traceback (most recent call last):
  File "orders.py", line 18, in save_order
    write_to_database(order)
  File "orders.py", line 42, in write_to_database
    raise ConnectionError("orders-db unreachable")
ConnectionError: orders-db unreachable
```

Practical takeaway: `logger.exception()` must be called from inside an except block -- it automatically grabs the exception currently being handled. Outside an except block, use `logger.error(..., exc_info=True)` instead.

9. Best Practices Checklist

Use `getLogger(__name__)`, not the root logger directly

Gives every module its own named logger, so you can tell instantly where a message came from and control modules independently.

Never log sensitive data

Customer phone numbers, payment details, and passwords should never appear in log files — those files are often less protected than the database.

Pass variables as arguments, not f-strings

`logger.info("Order %s", order_id)` avoids building the string when the log level would discard it anyway.

Set different levels for different environments

DEBUG while developing locally, INFO or WARNING in production — without changing a single log statement in the code.

Always use `logger.exception()` inside except blocks

A logged error with no traceback is far harder to debug than one with the full stack trace attached.

Rotate log files

Use `RotatingFileHandler` or `TimedRotatingFileHandler` for anything that runs repeatedly or continuously.

10. Common Mistakes

Mistake	Why it's a problem
Using <code>print()</code> in production code	No timestamps, no severity, no way to disable it without editing and redeploying code.
Calling <code>basicConfig()</code> more than once expecting it to reconfigure	Only the first call has any effect; later calls are silently ignored.
Logging everything at INFO level	Defeats the purpose of levels — you lose the ability to filter noise from real problems.
Catching an exception and logging only <code>logger.error("failed")</code>	No traceback means no idea which line failed or why — use <code>logger.exception()</code> instead.
Never rotating log files	Long-running or frequently-run jobs can grow log files until they fill the disk.

11. Cheat Sheet

Task	Code
Quick setup	<code>logging.basicConfig(level=logging.INFO)</code>
Get a module logger	<code>logger = logging.getLogger(__name__)</code>
Log at each level	<code>logger.debug() / .info() / .warning() / .error() / .critical()</code>
Log to a file	<code>logging.FileHandler("app.log")</code>
Rotate by size	<code>logging.handlers.RotatingFileHandler(...)</code>
Rotate by time	<code>logging.handlers.TimedRotatingFileHandler(...)</code>
Log an exception with traceback	<code>logger.exception("message")</code> (inside except block)
Custom message format	<code>logging.Formatter("%(asctime)s %(levelname)s %(message)s")</code>

Companion resource: pairs well with a hands-on notebook where students configure a logger for a small ShopEasy ETL script and intentionally trigger each severity level.